

NEX COMMUNITY PLATFORM

Technical Specification Document

Version 1.0

Prepared for the Nex Builder Team

September 2026

Table of Contents

- [1. System Overview](#)
- [2. Proposed Architecture](#)
- [3. Technology Stack](#)
- [4. Repository Structure](#)
- [5. User Roles](#)
- [6. Authentication System](#)
- [7. Database Schema](#)
- [8. Profile Database](#)
- [9. Community System](#)
- [10. Anonymous Posting](#)
- [11. Project System](#)
- [12. Teammate / Collaboration System](#)
- [13. Tester System](#)
- [14. Opportunity System](#)
- [15. Event System](#)
- [16. Notification System](#)
- [17. Search](#)
- [18. File Uploads](#)
- [19. API Structure](#)
- [20. Security Requirements](#)
- [21. Moderation System](#)
- [22. Development Workflow](#)
- [23. MVP Development Priority](#)
- [24. Non-Functional Requirements](#)
- [25. Testing](#)
- [26. Definition of Done](#)
- [27. Initial Builder Assignment](#)
- [28. MVP Architecture \(V1\)](#)

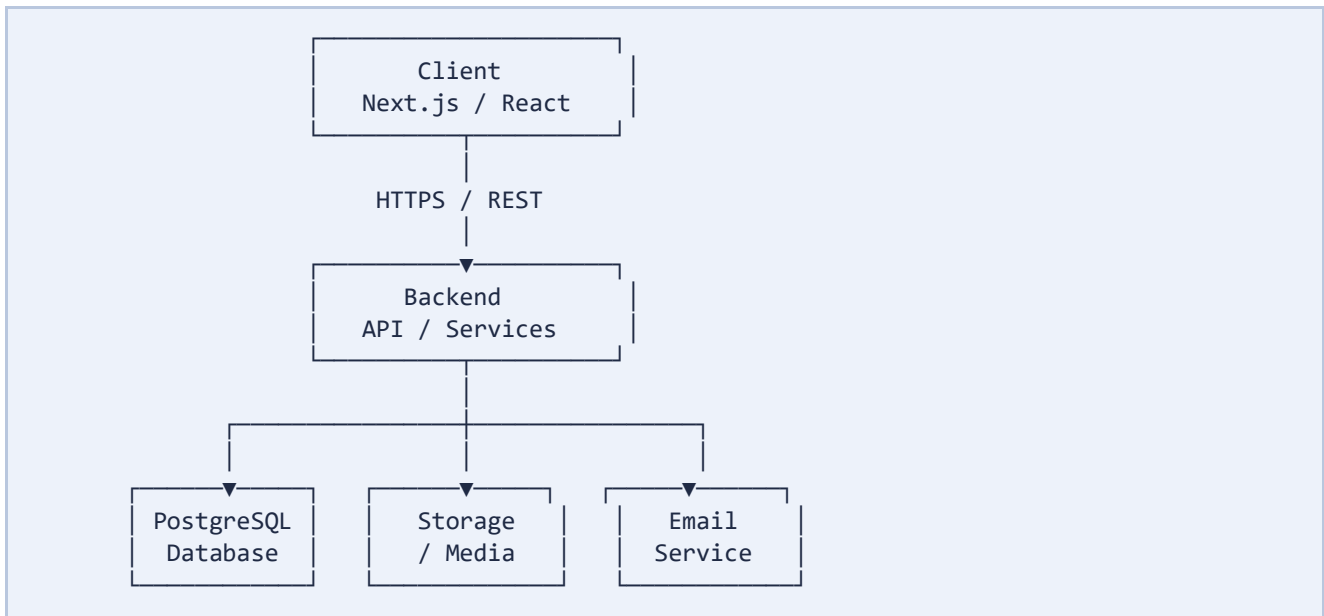
1. System Overview

Nex Community Platform is a web-based community application designed for students to:

- Communicate and participate in discussions
- Create and manage projects
- Find teammates and collaborators
- Find testers
- Discover opportunities
- Register for events
- Build a personal profile
- Share technical knowledge and resources

The platform will be developed incrementally using an MVP-first approach.

2. Proposed Architecture



3. Technology Stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

Backend

Recommended:

- Next.js API Routes / Route Handlers

Or, if the backend team wants a separate service:

- Node.js
- Express / Fastify
- TypeScript

Database

- PostgreSQL
- Supabase PostgreSQL

Authentication

Provider: Supabase Auth

Supported authentication methods:

- Email/password
- Google OAuth

Storage

Provider: Supabase Storage — used for:

- Profile pictures
- Project images
- Event images
- Attachments

Deployment

- Vercel — frontend/application
- Supabase — database, storage, authentication

Version Control

- Git
- GitHub

4. Repository Structure

Recommended monorepo structure:

```
nex-platform/  
├── app/  
│   ├── (auth)/  
│   ├── dashboard/  
│   ├── community/  
│   ├── projects/  
│   └── opportunities/
```

```

├── events/
├── profile/
├── admin/
├── components/
│   ├── ui/
│   ├── community/
│   ├── projects/
│   ├── opportunities/
│   └── events/
├── lib/
│   ├── auth/
│   ├── db/
│   ├── api/
│   └── utils/
├── types/
├── hooks/
├── public/
└── tests/

```

5. User Roles

Role	Permissions
Student (default)	Create posts · Comment · Create projects · Join projects · Request collaboration · Create tester requests · Register for events · Save opportunities · Manage profile
Moderator	Remove posts · Review reports · Moderate users · Manage community content
Administrator	Full platform management · User management · Event management · Opportunity management · Moderation · Platform configuration · Analytics

6. Authentication System

Registration

Required fields:

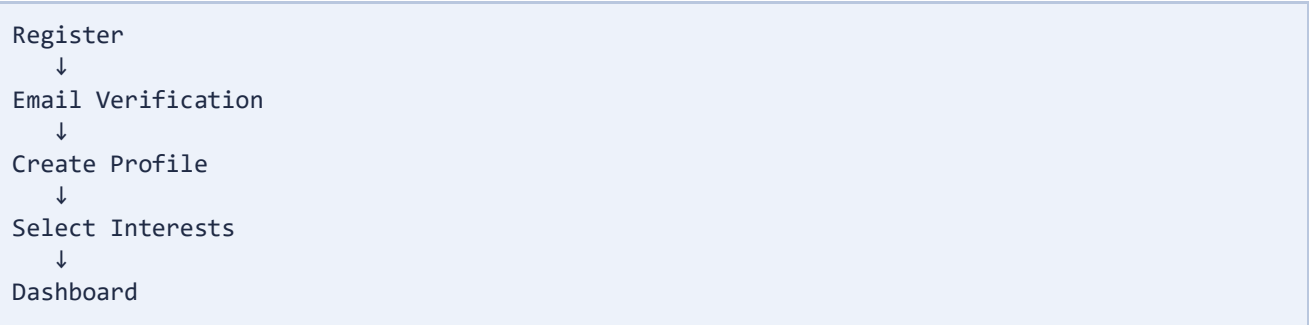
- Email
- Password
- First Name
- Last Name
- School
- Course
- Year Level

Optional fields:

- Profile Picture
- Bio

- Skills
- Interests
- Social Links

Authentication Flow



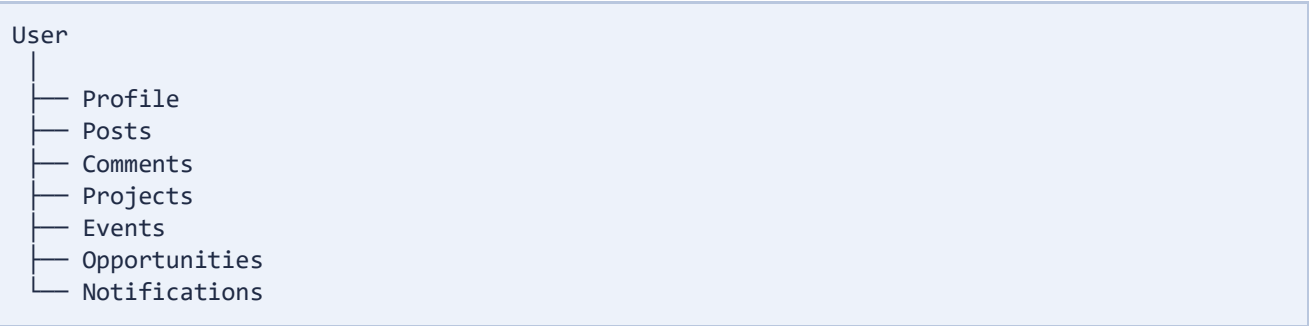
Authentication must use secure session management. Passwords must never be stored directly by the application.

7. Database Schema

Core entities:

users	posts	projects	tester_requests
profiles	comments	project_members	tester_signups
schools	reactions	project_requests	
skills	reports		
interests			
opportunities	events	notifications	resources
saved_opportunities	event_registrations		

Relationship Overview



8. Profile Database

Field	Notes
id	Primary key

Field	Notes
user_id	Foreign key → users
first_name	
last_name	
username	
school_id	Foreign key → schools
course	
year_level	
bio	
profile_image	
created_at	
updated_at	

Skills use a many-to-many relationship:

```

profiles
  |
profile_skills
  |
skills

```

9. Community System

Post

Field	Notes
id	Primary key
author_id	Foreign key → users
category_id	
title	
content	
post_type	See post types below
is_anonymous	
created_at	
updated_at	

Post types:

- DISCUSSION
- QUESTION
- PROJECT

- COLLABORATION
- OPPORTUNITY
- ANNOUNCEMENT

Comments

Field	Notes
id	Primary key
post_id	Foreign key → posts
author_id	Foreign key → users
content	
is_anonymous	
created_at	
updated_at	

10. Anonymous Posting

Anonymous posts hide the author's identity from normal users. The database must still retain:

- author_id
- is_anonymous = true

This allows moderators to identify the account when necessary.

Do not create separate anonymous accounts.

11. Project System

Project

Field	Notes
id	Primary key
owner_id	Foreign key → users
name	
description	
category	
status	See statuses below
repository_url	
demo_url	
created_at	
updated_at	

Statuses:

- IDEA
- PLANNING
- PROTOTYPE
- DEVELOPMENT
- COMPLETED
- ARCHIVED

Project Members

Field	Notes
id	Primary key
project_id	Foreign key → projects
user_id	Foreign key → users
role	
joined_at	

Project Requests

Field	Notes
id	Primary key
project_id	Foreign key → projects
user_id	Foreign key → users
message	
status	PENDING · ACCEPTED · REJECTED
created_at	

12. Teammate / Collaboration System

Users should be able to create collaboration requests.

Collaboration Requests

Field	Notes
id	Primary key
creator_id	Foreign key → users
project_id	Foreign key → projects
title	
description	
skills_required	

Field	Notes
status	
created_at	

Users can apply to a request.

Collaboration Applications

Field	Notes
id	Primary key
request_id	Foreign key → collaboration_requests
applicant_id	Foreign key → users
message	
status	
created_at	

13. Tester System

Students can create requests for project testing.

Tester Requests

Field	Notes
id	Primary key
project_id	Foreign key → projects
creator_id	Foreign key → users
title	
description	
number_needed	
estimated_time	
deadline	
created_at	

Tester Signups

Field	Notes
id	Primary key
tester_request_id	Foreign key → tester_requests
user_id	Foreign key → users
status	

Field	Notes
created_at	

14. Opportunity System

Field	Notes
id	Primary key
title	
description	
organizer	
category	See categories below
location	
is_online	
deadline	
eligibility	
application_url	
created_by	Foreign key → users
created_at	
updated_at	

Categories:

- HACKATHON
- IDEATHON
- COMPETITION
- INTERNSHIP
- SCHOLARSHIP
- FELLOWSHIP
- WORKSHOP
- CONFERENCE
- STARTUP

Saved Opportunities

Field	Notes
user_id	Foreign key → users
opportunity_id	Foreign key → opportunities
created_at	

15. Event System

Field	Notes
id	Primary key
title	
description	
location	
start_date	
end_date	
capacity	
registration_deadline	
image_url	
created_by	Foreign key → users
created_at	
updated_at	

Event Registration

Field	Notes
event_id	Foreign key → events
user_id	Foreign key → users
status	
registered_at	

16. Notification System

Notifications should be generated for:

- Comment replies
- Project requests
- Collaboration requests
- Event reminders
- Opportunity deadlines
- Announcements
- Moderator actions

Field	Notes
id	Primary key
user_id	Foreign key → users

Field	Notes
type	
title	
message	
reference_id	
is_read	
created_at	

17. Search

Search should eventually support:

- Users
- Projects
- Posts
- Opportunities
- Events
- Resources

Example:

Search: "Python"

Results:

People
Projects
Discussions
Opportunities
Resources

Initial implementation can use PostgreSQL full-text search. More advanced search can be added later.

18. File Uploads

Supported files should be restricted by type and size.

Upload Type	Allowed Formats
Profile image	JPG / PNG / WEBP
Project image	JPG / PNG / WEBP
Documents	PDF

Files should be stored using object storage rather than directly inside PostgreSQL.

19. API Structure

If using REST:

```

/api/auth
/api/users
/api/profiles

/api/posts
/api/comments
/api/reactions
/api/reports

/api/projects
/api/projects/:id/members
/api/projects/:id/requests

/api/testers

/api/opportunities
/api/opportunities/:id/save

/api/events
/api/events/:id/register

/api/notifications

```

Example — Projects resource:

```

GET    /api/projects
POST   /api/projects
GET    /api/projects/:id
PATCH /api/projects/:id
DELETE /api/projects/:id

```

20. Security Requirements

Minimum requirements:

- HTTPS
- Secure authentication
- Password hashing handled by authentication provider
- Row-level authorization
- Input validation
- Rate limiting
- File upload validation
- XSS protection
- CSRF protection where applicable
- SQL injection prevention
- Environment variables for secrets
- No API keys committed to GitHub

Never place DATABASE_PASSWORD, SUPABASE_SERVICE_ROLE_KEY, API_KEYS, or SMTP_PASSWORD inside the repository.

21. Moderation System

Users can report:

- Spam
- Harassment
- Scam
- Inappropriate content
- Misleading content
- Other

Reports

Field	Notes
id	Primary key
reporter_id	Foreign key → users
target_type	
target_id	
reason	
description	
status	
reviewed_by	Foreign key → users
created_at	

Moderation Flow

```

Pending
  ↓
Review
  ↓
Dismiss  or  Take Action
  
```

22. Development Workflow

All code should go through GitHub.

Recommended branch structure:

```

main
|
├── develop
  
```

- feature/auth
- feature/projects
- feature/opportunities
- feature/events

Developers should not directly push to main.

Workflow:

Issue → Create Branch → Develop → Test →
Pull Request → Code Review → Merge → Deploy

23. MVP Development Priority

Phase	Scope
Phase 1	Authentication · Profiles · Community · Posts · Comments
Phase 2	Projects · Project members · Collaboration requests
Phase 3	Opportunities · Events · Notifications
Phase 4	Tester system · Anonymous posting · Search
Phase 5 (Advanced)	Smart teammate matching · Recommendation system · Reputation · Portfolio generation · Mentorship matching · Analytics

24. Non-Functional Requirements

Performance

Target:

- Initial page load under 3 seconds under normal conditions
- API responses under 500ms for common operations
- Pagination for feeds and lists

Scalability

The architecture should allow future migration to:

- Dedicated backend
- Redis caching
- Background workers
- Search engine
- CDN
- Microservices if actually necessary

Do not implement microservices prematurely.

Availability

The platform should remain functional even if individual non-critical services fail.

25. Testing

Each feature should include:

Unit Tests

Test individual functions/components.

Integration Tests

```
graph TD; Frontend --> API; API --> Database;
```

Frontend
↓
API
↓
Database

End-to-End Tests

Important flows:

- Register
- Login
- Create Post
- Comment
- Create Project
- Join Project
- Register for Event
- Save Opportunity
- Report Content

26. Definition of Done

A feature is considered complete when:

- Feature requirements are implemented
- UI is responsive
- Authentication/authorization works
- Input validation exists
- Error states are handled
- Tests are written
- Code has been reviewed
- Documentation is updated
- Feature is deployed successfully

27. Initial Builder Assignment

For a learning-oriented team:

Track	Suggested Headcount
Frontend	2–3 builders
Backend	2 builders
Database / DevOps	1–2 builders
Full-stack / Integration	1 builder

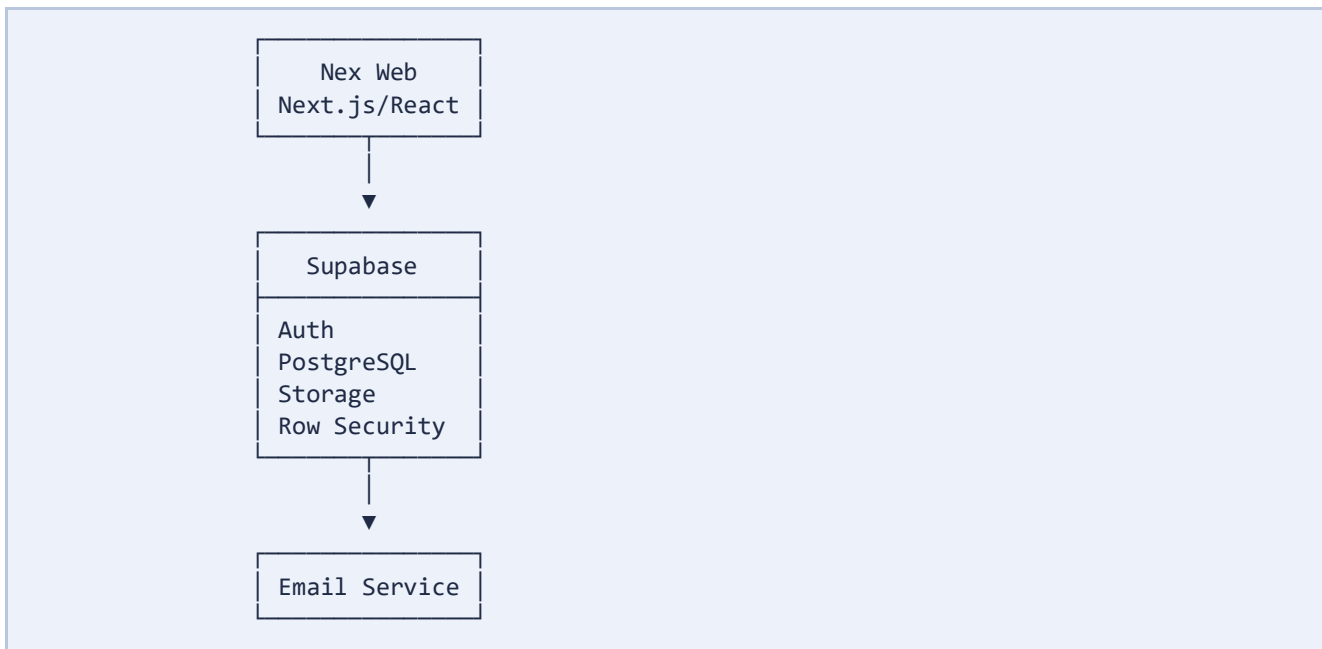
Each team should have a lead who can mentor the others.

The goal isn't simply to finish the platform quickly. The goal is for every builder to experience a real development workflow:

Issue → Code → Review → Test → Deploy → Feedback → Iterate

28. MVP Architecture (V1)

Keep V1 relatively simple:



This is enough to get Nex's first real version running without overengineering it.